

SEEK WANDER

Curated Luxury Journeys

Security Architecture & Data Protection

Platform Integrity Report, Confidential

March 2026

Executive Summary

At Seek Wander, the trust of our guests is the foundation upon which every journey is built. Our security architecture is designed with the same meticulous attention to detail as the itineraries we craft: layered, deliberate, and invisible to the traveller.

Every layer of the platform, from credential storage to database ownership checks, has been designed to protect guest data, prevent unauthorised access, and ensure that the experience of using Seek Wander feels effortless because the security work happens silently in the background.

The following report documents our security posture as of March 2026, following a 33-point architectural audit. All critical findings have been resolved and verified.

Security Posture at a Glance

9 Active Security Defences	5 / Hour Rate Limit Per User
0 Secret Keys Exposed to Browser	100% Routes with Auth Enforcement

I. Infrastructure & Data Protection

API Key Architecture

- All secret credentials (Anthropic AI, Google Maps Server Key, Clerk Secret, Database URLs, Upstash Redis tokens) are stored exclusively as server-side environment variables. They are never exposed in client-side JavaScript bundles.
- A strict split is enforced: NEXT_PUBLIC_ prefixed keys are browser-safe by design and restricted by domain. All other keys are server-only and inaccessible to the client runtime.
- Verified: Zero secret keys found in client-side JavaScript bundles during the March 2026 security audit.

Google Maps Key Hardening

- The browser-facing Maps key is restricted by HTTP Referrer to two domains only: the production Vercel deployment and localhost for development.
- API-level restrictions limit the key to Maps JavaScript API and Places API exclusively, preventing use for any other Google service even if the key were obtained.
- The server-side Maps key carries no HTTP referrer restriction (servers have no referrer header) and is isolated to server-side API routes only, never transmitted to the client.

Environment & Version Control Isolation

- The .env.local file is confirmed absent from all git history, verified via full history search with zero results. Sensitive configuration has never been committed.
- Sensitive configuration is managed exclusively through the deployment platform's encrypted environment variable storage, with access controlled at the account level.

II. Authentication & Access Control

Identity & Session Management

- Authentication is powered by Clerk, a dedicated, enterprise-grade identity provider. Session tokens are managed server-side and never reconstructed client-side.
- All protected API routes enforce authentication as the first operation, before any database query or business logic executes. There is no path to data without a valid session.
- Unauthenticated requests to `/api/trips` and `/api/trips/[id]` receive an immediate 401 Unauthorized response without revealing any information about the data.

IDOR Prevention (Insecure Direct Object Reference)

- Every data access operation enforces both record existence and ownership simultaneously. A user cannot access, view, or delete another guest's itinerary, even with a valid session token and a known trip ID.
- Missing records and unauthorised access return an identical neutral 404 response, preventing attackers from enumerating which trip IDs exist in the system.
- Trip deletion uses an atomic database-level compound constraint (`deleteMany` with both `id` AND `userId`), eliminating any time-of-check to time-of-use race window between the ownership read and the delete operation.

Admin Access Control

- The `/admin/metrics` dashboard is protected by a server-side environment variable comparison. Non-matching sessions receive a neutral 404. Route existence is not disclosed to any unauthorised party.

III. Application Security

HTTP Security Headers

X-Frame-Options	DENY	Prevents clickjacking attacks
X-Content-Type-Options	nosniff	Blocks MIME-type sniffing exploits
Referrer-Policy	strict-origin-when-cross-origin	Limits referrer data leakage
Permissions-Policy	camera=(), microphone=(), payment=(), usb=()	Blocks hardware API access

Input Validation & Prompt Injection Defence

- All API POST bodies are validated against strict Zod schemas before any processing. Malformed or unexpected payloads receive a structured 400 response, never a raw server error or stack trace.
- The AI generation system prompt is delivered as a fixed system-role message. User-supplied content (destination names, dietary preferences, hotel addresses) cannot override or modify the model's behaviour.
- Hotel names and addresses are validated with a blocklist regex rejecting shell metacharacters and prompt-injection control characters, while permitting all valid international address characters (Arabic script, CJK, accented Latin).

Rate Limiting

- A sliding-window rate limiter (Upstash Redis) enforces a maximum of 5 requests per hour per user across the itinerary generation API.
- Authenticated users are keyed by their unique Clerk userId. Unauthenticated requests are keyed by IP address. Requests with neither identifier are rejected with a 400 before reaching the limiter, preventing shared anonymous bucket abuse.

Dependency Security

- The full dependency tree has been audited. The flatted vulnerability (prototype pollution + DoS) has been patched.

— Remaining Next.js framework CVEs are tracked and scheduled for resolution with the planned Next.js 15 + Clerk v7 upgrade.

IV. Background Job Security

Automated Garbage Collection

- A nightly Vercel Cron job clears stale place cache records older than 14 days. The route is protected by a Bearer token secret. Requests without the correct Authorization header receive an immediate 401 Unauthorized response.
- The PlaceCache table carries a database index on the updatedAt column, ensuring the nightly deletion is an $O(\log n)$ indexed range scan rather than a full-table sequential scan. Recently accessed records are automatically preserved.
- The cron secret is generated via cryptographically secure random generation and stored as a server-side environment variable, rotatable without code changes.

Photo Reference Architecture

- Cached place data stores raw photo reference tokens rather than full API key-embedded URLs. Photo URLs are reconstructed at serve time using the current server key, making all cached data resilient to API key rotation with no data migration required.
- Old-format records (stored before this architecture was introduced) are detected via an isOldFormat flag and treated as expired. They self-refresh on the next cache miss with zero manual intervention required.
- This architecture means an API key rotation can be completed in under 60 seconds with no user-visible disruption and no stale credential exposure in any stored data.

Cost & Usage Observability

- Every itinerary generation is logged to a CostLog table with per-request AI token costs and Google Places API call costs, broken down to six decimal places.
- Cache hit ratios are tracked per generation, providing visibility into how effectively the PlaceCache is reducing third-party API exposure.
- Cost data is accessible only via the admin dashboard, which is protected by the ADMIN_USER_ID ownership check. It is never transmitted to the client.

V. Continuous Monitoring & Security Roadmap

Current Monitoring

- Rate limit analytics are surfaced in the Upstash dashboard, providing real-time visibility into per-user request patterns and potential abuse attempts.
- Vercel deployment logs capture all server-side errors with structured context for post-incident review and root cause analysis.
- The /admin/metrics internal dashboard provides a live view of generation costs, cache performance, and API usage across all platform users.

Security Roadmap

Sentry Real-Time Error Monitoring

IN PROGRESS

Integration of Sentry for automatic error capture, alerting, and anomaly detection across all API routes and client-side interactions. Enables proactive incident response before users are affected.

Next.js 15 + Clerk v7 Framework Upgrade

PLANNED

Scheduled upgrade to patch underlying framework CVEs identified in Next.js 14.x. Tied to Clerk v7 compatibility. Both will be upgraded together in a single coordinated release to maintain consistency.

March 2026 Security Audit Baseline

A 33-point system audit was completed covering all authentication, authorisation, input validation, credential management, and data isolation controls. All 9 identified findings have been resolved and verified. No regressions were introduced by subsequent changes.

SEEK WANDER

Curated Luxury Journeys

“Security is not a feature. It is the foundation.”

This document reflects the security posture of the Seek Wander platform as of March 2026. Architecture decisions are reviewed continuously and this document is updated to reflect all material changes.

For security disclosures or technical enquiries, please contact the engineering team directly.